

Show understanding of the relationship between assembly language and machine code

9608_s21_qp_11_sc_3.c.ii

QUESTION

- (ii) State the **maximum** number of op codes that can be represented using eight bits.
 [1]

MARK SCHEME

3(c)(ii)	256	1
----------	-----	---

9608_w19_qp_11_sc_1.c

QUESTION

- (c) The following statements describe features of a low-level language.
 Complete the statements by writing the appropriate terms in the spaces.
- A is a sequence of instructions that are given an identifier. These instructions may need to be executed several times.
- A is an instruction that tells the assembler to do something. It is not a program instruction.
- The processor's instruction set can be put into several groups. One of these groups is [3]

MARK SCHEME

1(c)	1 mark for each correctly inserted term A macro is a sequence of instructions that are given an identifier. These instructions may need to be executed several times. A directive is an instruction that tells the assembler to do something. It is not a program instruction. The processor's instruction set can be put into several groups. One of these groups is data movement // input and output // arithmetic operations // jump instructions // compare instructions // modes of addressing.	3
------	---	---

QUESTION

The following shows two of the program instructions in machine code.

Instruction	Machine code	
	Binary	Hexadecimal
OUTCH	1100 0111	C7
JNE CarryOn	A	B

Each of the numbers **A** and **B** represents the complete instruction in two bytes, one byte for the op code and one byte for the operand.

(ii) Use the following instruction set to write the numbers for **A** and **B**.

A (binary)

B (hexadecimal)

[3]

MARK SCHEME

4(d)(ii)	A – 1110 0110 0110 1000 (1) (1)	3
	B – E6 68 (1)	

QUESTION

(c) An assembly language program can contain both **macros** and **directives**.

(i) Explain what is meant by these terms.

Macro

.....

.....

Directive

.....

.....

[3]

MARK SCHEME

4(c)(i)	1 Mark per bullet Maximum 2 for Macro Maximum 2 for Directive Maximum 3 in total Macro A group of instructions given a name // subroutine A group of instructions that need to be executed several times within the same program The statements are written once and called using the name whenever they need to be executed Macro code is inserted into the source file at each place it is called By example Directive An instruction that directs the assembler to do something A directive is not a program instruction It is information for the assembler By example	3
---------	--	----------

9608_w17_qp_12_sc_4.c.ii

QUESTION

(ii) Give an example of the use of a directive.

.....
.....[1]

MARK SCHEME

4(c)(ii)	1 Mark for a suitable example For example: State the start address for the program //tell the assembler to set aside space for variables // include an external file etc.	1
----------	--	----------

9608_s17_qp_13_sc_4.b

QUESTION

503 | 502

- (b) Each machine code instruction is encoded as 16-bits (8-bit op code followed by an 8-bit operand).

Write the machine code for the following instructions:

LDM #17

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

LDX #97

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

[3]

MARK SCHEME

4(b)	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr></table>	0	0	0	0	0	0	0	1	0	0	0	0	0	1	1	0	<table><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	0	0	0	1	0	0	0	1	0	1	1	0	0	0	0	1		3
0	0	0	0	0	0	0	1																													
0	0	0	0	0	1	1	0																													
0	0	0	1	0	0	0	1																													
0	1	1	0	0	0	0	1																													
	Both correct op codes			1																																
	Operand 0001 0001			1																																
	Operand 0110 0001			1																																

9608_s17_qp_13_sc_4.c

QUESTION

- (c) Using an 8-bit operand, state the maximum number of memory locations, in denary, that can be directly addressed.

.....[1]

MARK SCHEME

4(c)	256	1
------	-----	----------

9608_s17_qp_13_sc_4.d.ii

QUESTION

- (ii) A second instruction has been written in hexadecimal as:

05 3F

Write the equivalent assembly language instruction, with the operand in denary.

.....[2]

MARK SCHEME

4(d)(ii)	LDI 63	1 1	2
	LDI 63		

9608_s17_qp_12_sc_5.b

QUESTION

		357	22
		358	

(b) Each machine code instruction is encoded as 16 bits (8-bit op code followed by an 8-bit operand).

Write the machine code for these instructions:

LDM #67

LDX #7

[3]

MARK SCHEME

5(b)	Op code								Operand								3	
	0	0	0	1	0	0	1	0	0	1	0	0	0	0	1	1		
	0	0	0	1	0	1	0	1	0	0	0	0	0	1	1	1		
	Both correct op codes																	1
	Operand 0100 0011																	1
	Operand 0000 0111																	1

9608_s17_qp_12_sc_5.c.ii

QUESTION

(ii) A second instruction has been written in hexadecimal as:

16 4D

Write the assembly language for this instruction with the operand in denary.

.....[2]

MARK SCHEME

5(c)(ii)	LDR #77		2
	LDR #77	1 1	

9608_s17_qp_11_sc_4.b

QUESTION

503 502

(b) Each machine code instruction is encoded as 16-bits (8-bit op code followed by an 8-bit operand).

Write the machine code for the following instructions:

LDM #17

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

LDX #97

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

[3]

MARK SCHEME

4(b)	<table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr></table>	0	0	0	0	0	0	0	1	0	0	0	0	0	1	1	0	<table><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr></table>	0	0	0	1	0	0	0	1	0	1	1	0	0	0	0	1	
0	0	0	0	0	0	0	1																												
0	0	0	0	0	1	1	0																												
0	0	0	1	0	0	0	1																												
0	1	1	0	0	0	0	1																												
	Both correct op codes		1																																
	Operand 0001 0001		1																																
	Operand 0110 0001		1																																

9608_s17_qp_11_sc_4.c

QUESTION

- (c) Using an 8-bit operand, state the maximum number of memory locations, in denary, that can be directly addressed.

.....[1]

MARK SCHEME

4(c)	256	1
------	-----	----------

9608_s17_qp_11_sc_4.d.ii

QUESTION

- (ii) A second instruction has been written in hexadecimal as:

05 3F

Write the equivalent assembly language instruction, with the operand in denary.

.....[2]

MARK SCHEME

4(d)(ii)	LDI 63	2
	LDI 63	1 1

Describe the different stages of the assembly process for a two-pass assembler

9608_s20_qp_12_sc_5.a

QUESTION

- 5 (a) The steps 1 to 6 describe the first pass of a two-pass assembler.

The following three statements are used to complete the sequence of steps.

A	If it is already in the symbol table, it checks to see if the absolute address is known
B	When it meets a symbolic address, it checks to see if it is already in the symbol table
C	If it is known, it is entered

Write one of the letters **A**, **B** or **C** in the appropriate step to complete the sequence.

1. The assembler reads the assembly language instructions
2.
3. If it is not, it adds it to the symbol table
4.
5.
6. If it is not known, it is marked as unknown.

[2]

MARK SCHEME

5(a)	1 mark for one letter in the correct place, 2 marks for all three correct	2
	2 B	
	4 A	
	5 C	

QUESTION

- 5 A simple program written in assembly language is translated using a two-pass assembler.

- (a) The table contains some of the tasks performed by a two-pass assembler.

Tick (✓) **one** box in each row to indicate whether the task is performed at the first or second pass. The first row has been completed for you.

Task	First pass	Second pass
Creation of symbol table	✓	
Expansion of macros		
Generation of object code		
Removal of comments		

[2]

MARK SCHEME

5(a)	1 mark for two correct ticks, 2 marks for three correct ticks	2															
	<table> <tr> <th>Task</th><th>First pass</th><th>Second pass</th></tr> <tr> <td>Creation of symbol table</td><td>✓</td><td></td></tr> <tr> <td>Expansion of macros</td><td>✓</td><td></td></tr> <tr> <td>Generation of object code</td><td></td><td>✓</td></tr> <tr> <td>Removal of comments</td><td>✓</td><td></td></tr> </table>	Task	First pass	Second pass	Creation of symbol table	✓		Expansion of macros	✓		Generation of object code		✓	Removal of comments	✓		
Task	First pass	Second pass															
Creation of symbol table	✓																
Expansion of macros	✓																
Generation of object code		✓															
Removal of comments	✓																

QUESTION

(c) The program given in **part (b)** is to be translated using a two-pass assembler.

The program has been copied here for you. The program now starts with a directive which tells the assembler to load the first instruction of the program to address 100.

Label	
	ORG #0100
StartProg:	LDV #CountDown
	CMP Num1
	JNE CarryOn
	JMP Finish
CarryOn:	OUTCH
	LDD CountDown
	DEC
	STO CountDown
	JMP StartProg
Finish:	LDM #88
	OUTCH
	END
CountDown:	15
	32
	51
	67
Num1:	32

On the first pass of the two-pass process, the assembler adds entries to a symbol table.

The following symbol table shows the first eleven entries, part way through the first pass.

The circular labels show the order in which the assembler made the entries to the symbol table.

Symbol table

Symbolic address		Absolute address	
StartProg	1	100	2
CountDown	3	UNKNOWN	4
Num1	5	UNKNOWN	6
CarryOn	7	UNKNOWN	8 104 11
Finish	9	UNKNOWN	10

Explain how the assembler made these entries to the symbol table.

.....

.....

.....

.....

.....

.....[3]

MARK SCHEME

4(c)	Three marks from: The assembler scans the assembly language instructions in sequence When it meets a symbolic address checks to see if already in symbol table If not, it adds it to the symbol table in the symbolic address column If it is already in symbol table check if absolute address known If the absolute address is known, it is entered in the appropriate cell If the absolute address is not known mark / leave as unknown	Max 3
------	--	--------------

QUESTION

(d) The assembler software must then complete the second pass building up the executable file.

(i) Name the second table needed when the assembler software carries out the second pass.

.....[1]

MARK SCHEME

4(d)(i)	The op code / mnemonic / instruction table	1
---------	--	---

QUESTION

- (e) The program given in **part (d)** is to be translated using a two-pass assembler. The program has been copied here for you.

Label	Instruction
StartProg:	LDV #Offset
	CMP Value
	JPE EndProg
	OUTCH
	LDD Offset
	INC
	STO Offset
	JMP StartProg
EndProg:	END
Offset:	10
	50
	65
	89
	32
Value:	32

On the first pass, the assembly process adds entries to a symbol table.

The following symbol table shows the first five entries, part way through the first pass.

The circular labels show the order in which the assembler made the entries to the symbol table.

Complete the symbol table. Use circular labels to show the order in which the assembler makes the entries.

Symbol table

Symbolic address	Relative address
StartProg (1)	0 (2)
Offset (3)	UNKNOWN (4)
Value (5)	

[6]

MARK SCHEME

4(e)

Mark as follows:

Table entries:

1 Mark per bullet, max 4

EndProg

2 · Unknown

9

14

8

Numbering:

1 Mark per bullet, max 2

Relative address of Value is numbered 6

Number given for EndProg is next number in sequence to relative address of Value

All numbers correct – award 2 marks

Symbolic address	Relative address
<u>StartProg</u>	0
Offset	UNKNOWN 9 (1)
Value	UNKNOWN (6) 14 (1)
<u>EndProg</u> (7)	UNKNOWN (8) 8 (9)

6

QUESTION

- 2 Assemblers translate from assembly language to machine code. Some assemblers scan the assembly language program twice; these are referred to as two-pass assemblers.

The following table shows five activities performed by two-pass assemblers.

Write 1 or 2 to indicate whether the activity is carried out during the first pass or during the second pass.

Activity	First pass or second pass
any symbolic address is replaced by an absolute address	
any directives are acted upon	
any symbolic address is added to the symbolic address table	
data items are converted into their binary equivalent	
forward references are resolved	

[5]

MARK SCHEME

2

Activity	First pass or second pass
any symbolic address is replaced by an absolute address	2
any directives are acted upon	1
any symbolic address is added to the symbolic address table	1
data items are converted into their binary equivalent	1
forward references are resolved	2

[5]

Trace a given simple assembly language program

9608_w21_qp_11_sc_8

QUESTION

- 8 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to the ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to the ACC
STO	<address>	Store the contents of the ACC at the given address
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address

The current contents of the main memory are:

Address	Instruction
130	LDI 160
131	ADD 153
132	STO 153
133	LDD 160
134	INC ACC
135	STO 160
136	JMP 130
...	
150	13
151	23
152	11
153	0
...	
160	150

Complete the trace table for **two iterations** of the loop.

[illegible]

[4]

MARK SCHEME

8

One mark for each set of rows (shaded and unshaded)

4

Instruction address	ACC	Memory address				
		150	151	152	153	160
		13	23	11	0	150
130	13					
131	13					
132					13	
133	150					
134	151					
135						151
136						
130	23					
131	36					
132					36	
133	151					
134	152					
135						152
136						

QUESTION

- 3 The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
JMP	<address>	Jump to the given address.
CMP	<address>	Compare the contents of ACC with the contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The current contents of the main memory and selected values from the ASCII character set are:

Address	Instruction
75	LDR #2
76	LDX 180
77	CMP #0
78	JPE 82
79	OUT
80	DEC IX
81	JMP 76
82	END
...	⌋
180	0
181	41
182	71
183	40
184	70
185	43
186	69

ASCII code table (selected codes only)

ASCII code	Character
36	\$
40	(
41	)
43	+
69	E
70	F
71	G

(a) Complete a trace table for the execution of the program.

[illegible]

[5]

MARK SCHEME

3(a)

1 mark per shaded section

5

Instruction Address	IX	ACC	Memory Address							Output
			180	181	182	183	184	185	186	
			0	41	71	40	70	43	69	
75	2									
76		71								
77										
78										
79										G
80	1									
81										
76		41								
77										
78										
79										)
80	0									
81										
76		0								
77										
78										
82										

QUESTION

- 3 The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
STO	<address>	Store the contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
JMP	<address>	Jump to the given address.
CMP	<address>	Compare the contents of ACC with the contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
END		Return control to the operating system.

The current contents of the main memory are:

Address	Instruction
50	LDI 103
51	CMP 101
52	JPE 59
53	ADD 102
54	STO 102
55	LDD 100
56	INC ACC
57	STO 100
58	JMP 51
59	ADD 102
60	STO 102
61	END
...	
100	1
101	3
102	0
103	100

(a) Trace the program currently in memory using the following trace table.

[illegible]

[5]

MARK SCHEME

3(a)

1 Mark for each set of shaded rows

5

Instruction Address	ACC	Memory Address			
		100	101	102	103
		1	3	0	100
50	1				
51					
52					
53	1				
54				1	
55	1				
56	2				
57		2			
58					
51					
52					
53	3				
54				3	
55	2				
56	3				
57		3			
58					
51					
52					
59	6				
60				6	
61					

QUESTION

- 3 The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the content of the location at the given address to ACC.
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare the contents of ACC with the contents of <address>.
JMP	<address>	Jump to the given address.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
STO	<address>	Store the contents of ACC at the given address.
END		Return control to the operating system.

The current contents of the main memory are:

Address	Instruction
100	LDD 200
101	CMP 201
102	JPE 106
103	DEC ACC
104	STO 200
105	JMP 101
106	END
...	
200	2
201	0
202	200

(a) Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address		
		200	201	202
		2	0	200

[3]

MARK SCHEME

3(a)

1 mark for each set of shaded rows

3

Instruction Address	ACC	Memory Address		
		200	201	202
		2	0	200
100	2			
101				
102				
103	1			
104		1		
105				
101				
102				
103	0			
104		0		
105				
101				
102				
106				

QUESTION

(c) The table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDM	#n	Immediate addressing. Load the denary number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the denary number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare contents of the address given with the contents of ACC
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to screen the character whose ASCII value is stored in ACC
END		Return control to the operating system

Complete the trace table for the following assembly language program. The first instruction has been completed for you.

Address	Instruction
20	LDD 103
21	CMP 101
22	JPE 30
23	LDD 100
24	ADD 101
25	STO 100
26	LDD 103
27	INC ACC
28	STO 103
29	JMP 20
30	END
...	
100	1
101	2
102	3
103	0

[illegible]

[6]

MARK SCHEME

5(c)

1 mark for each highlighted section block

6

Instruction address	ACC	Memory address			
		100	101	102	103
		1	2	3	0
20	0				
21					
22					
23	1				
24	3				
25		3			
26	0				
27	1				
28					1
29					
20	1				
21					
22					
23	3				
24	5				
25		5			
26	1				
27	2				
28					2
29					
20	2				
21					
22					
30					

QUESTION

(c) The following table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDM	#n	Immediate addressing. Load the denary number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the denary number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare contents of ACC with denary number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system

The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are:

Address	Instruction
---------	-------------

50	LDM #0
51	STO 401
52	LDX 300
53	CMP #0
54	JPE 62
55	ADD 400
56	OUT
57	LDD 401
58	INC ACC
59	STO 401
60	INC IX
61	JMP 52
62	END
...	
300	2
301	5
302	0
303	4
...	
400	64
401	
IX	0

ASCII code	Character
------------	-----------

65	A
66	B
67	C
68	D
69	E

Trace the program currently in memory using the following trace table. The first instruction has been completed for you.

[illegible]

[8]

MARK SCHEME

3(c)	1 mark per bullet • Storing 0 in 401 (line 51) • Loading memory location 300, value 2 to ACC (line 52) • Adding 64 to ACC to give 66 (line 55) • Outputting B (line 56) • Load 0 (line 57), increment ACC (line 58) and store 1 in 401 (line 59) • Incrementing IX (line 60) • Loading 5 (line 52), adding 64 (line 55), outputting E (line 56) loading 1 (line 57), incrementing ACC (line 58), storing 2 in 401 (line 59) and incrementing IX (line 60) • Load 0 (line 52) <u>and end</u>	8
------	--	----------

QUESTION

- (c) The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDM	#n	Immediate addressing. Load the denary number n to ACC.
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the denary number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
CMP	#n	Compare contents of ACC with denary number n.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are:

Address Instruction

20	LDM #0
21	STO 300
22	CMP #0
23	JPE 28
24	LDX 100
25	ADD 301
26	OUT
27	JMP 30
28	LDX 100
29	OUT
30	LDD 300
31	INC ACC
32	STO 300
33	INC IX
34	CMP #2
35	JPN 22
36	END
...	
100	65
101	67
102	69
103	69
104	68
...	
300	
301	33
IX	0

ASCII code table (Selected codes only)

ASCII Code	Character
65	A
66	B
67	C
68	D
69	E
97	a
98	b
99	c
100	d
101	e

Trace the program currently in memory using the following trace table. The first instruction has been completed for you.

[illegible]

[8]

MARK SCHEME

5(c)	1 mark per bullet point Storing 0 in 300 (line 21) Loading 65 (line 28) Outputting A (line 29) Loading 0 (line 30), incrementing ACC (line 31) and storing in 300 (line 32) Incrementing IX (line 33) Loading 67 (line 24) and adding 33 (line (25) Outputting d (line 26) Loading 1 (line 30), incrementing ACC (line 31), storing in 300 (line 32) and incrementing IX (line 33)	8
------	---	----------

QUESTION

- (c) The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are provided with a copy of the instruction set.

Address Instruction

20	LDD 96
21	CMP 97
22	JPE 32
23	LDX 86
24	CMP 98
25	JPN 27
26	OUT
27	LDD 96
28	INC ACC
29	STO 96
30	INC IX
31	JMP 21
32	END
...	
93	453
94	453
95	452
96	8
97	10
98	453

IX

ASCII code table (selected codes only)

ASCII code	Character
450	<
451	>
452	=
453	&
454	(
455	)

Instruction set

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

Complete the trace table for the given assembly language program.

[illegible]

[7]

MARK SCHEME

2(c)	1 mark per bullet point • Loading 8 (instruction 20) • Comparison and loading 453 (instructions 21–23) • Outputting & (instruction 26) • Loading, incrementing and storing in 96 (instructions 27–29) • Incrementing Index Register (instruction 30) • Jumping and loading 452 (instruction 23) • Jumping, loading, incrementing, storing in 96, incrementing IX and end (instructions 24–32)	7
------	--	----------

QUESTION

- (d) The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are:

Address	Instruction
40	LDD 100
41	CMP 104
42	JPE 54
43	LDX 100
44	CMP 105
45	JPN 47
46	OUT
47	LDD 100
48	DEC ACC
49	STO 100
50	INC IX
51	JMP 41
52	
53	
54	END
...	
100	2
101	302
102	303
103	303
104	0
105	303

ASCII code	Character
300	/
301	*
302	-
303	+
304	^
305	=

IX

This is a copy of the instruction set.

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

Complete the trace table for the given assembly language program.

[illegible]

[7]

MARK SCHEME

3(d)	1 mark per bullet point • Loading 2, comparing with 104 (instructions 40 and 41) • Loading 302 (instruction 43) • Comparing, and branching to 47 (instructions 44, 45) • Loading, decrementing accumulator and storing (instructions 47, 48 and 49) • Incrementing Index Register (instruction 50) • Loading 303, comparing and outputting + (instructions 43–46) • Loading, decrementing accumulator and storing, incrementing Index Register and end (instructions 47–51, 41, 42 and 54)	7
------	---	----------

Instruction address	ACC	Memory address						IX	OUTPUT
		100	101	102	103	104	105		
		2	302	303	303	0	303	1	
40	2								
41									
43	302								
44									
45									
47	2								
48	1								
49		1							
50								2	
51									
41									
42									
43	303								
44									
45									
46									+
47	1								
48	0								
49		0							
50								3	
51									
41									
54									

QUESTION

- (d) The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are provided with a copy of the instruction set.

Address Instruction

70	LDX 200
71	OUT
72	STO 203
73	LDD 204
74	INC ACC
75	STO 204
76	INC IX
77	LDX 200
78	CMP 203
79	JPN 81
80	OUT
81	LDD 204
82	CMP 205
83	JPN 74
84	END
...	
200	130
201	133
202	130
203	0
204	0
205	2
IX	0

ASCII code table (selected codes only)

ASCII code	Character
127	?
128	!
129	"
130	*
131	\$
132	&
133	%
134	/

Instruction set

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

[illegible]

MARK SCHEME

4(d)	1 mark per bullet point - Outputting * (instruction 71) - Storing 130 in 203 (instruction 72) - Loading, incrementing accumulator and storing in 204 (instructions 73, 74 and 75) - Incrementing Index Register (instruction 76) - Loading 133, comparing and jumping to 81 (instructions 77, 78 and 79) - Loading, comparing and jumping to 74 (instructions 81, 82 and 83) - Incrementing accumulator, storing in 204 and incrementing index register, loading 130 (instructions 74–79) - Outputting * to end (instructions 80–84)	8
------	--	---

9608_w17_qp_13_sc_4.b

QUESTION

- (b) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Op code (binary)	Explanation
Op code (mnemonic)	Operand		
LDM	#n	1100 0001	Immediate addressing. Load number <i>n</i> to ACC.
LDD	<address>	1100 0010	Direct addressing. Load the contents of the given address to ACC.
LDV	#n	1100 0011	Relative addressing. Move to the address <i>n</i> locations from the address of the current instruction. Load the contents of this address to ACC.
STO	<address>	1100 0100	Store the contents of ACC at the given address.
DEC		1100 0101	Decrement the contents of ACC.
OUTCH		1100 0111	Output the character corresponding to the ASCII character code in ACC.
JNE	<address>	1110 0110	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	1110 0011	(Unconditionally) jump to the given address.
CMP	#n	1110 0100	Compare the contents of ACC with number <i>n</i> .

Complete the trace table for the following assembly language program.

Label	Instruction
StartProg:	LDV #CountDown
	CMP Num1
	JNE CarryOn
	JMP Finish
CarryOn:	OUTCH
	LDD CountDown
	DEC
	STO CountDown
	JMP StartProg
Finish:	LDM #88
	OUTCH
	END
CountDown:	15
	32
	51
	67
Num1:	32

ASCII code table (selected codes only)				
<Space>	3	B	C	X
32	51	66	67	88

Trace table:

ACC	CountDown	OUTPUT
	15	
67		C
15		

[5]

MARK SCHEME

4(b)	ACC	CountDown	OUTPUT	(1) (1) + (1) (1) (1)	}		5
		15					
	67		C				
	15						
	14	14					
	51		3				
	14						
	13	13					
	32						
	88		x				

9608_w17_qp_12_sc_4.d

QUESTION

- (d) The following table shows part of the instruction set for a processor. The processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code (mnemonic)	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDV	#n	Relative addressing. Move to the address n locations from the address of the current instruction. Load the contents of this address to ACC.
STO	<address>	Store the contents of ACC at the given address.
INC		Increment the contents of ACC.
OUTCH		Output the character corresponding to the ASCII character code in ACC.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JMP	<address>	Jump to the given address.
CMP	#n	Compare the contents of ACC with number n.

Complete the trace table for the following assembly language program.

Label	Instruction
StartProg:	LDV #Offset
	CMP Value
	JPE EndProg
	OUTCH
	LDD Offset
	INC
	STO Offset
	JMP StartProg
EndProg:	END
Offset:	10
	50
	65
	89
	32
Value:	32

ASCII code table (selected codes only)				
<Space>	2	A	B	Y
32	50	65	66	89

Trace table:

ACC	Offset	OUTPUT
	10	
50		2
10		

[5]

MARK SCHEME

4(d)	Mark as shown <table> <tr> <th>ACC</th><th>Offset</th><th>OUTPUT</th></tr> <tr> <td></td><td>10</td><td></td></tr> <tr> <td>50</td><td></td><td>2</td></tr> <tr> <td>10</td><td></td><td></td></tr> <tr> <td>11</td><td>11</td><td></td></tr> <tr> <td>65</td><td></td><td></td></tr> <tr> <td></td><td></td><td>A</td></tr> <tr> <td>11</td><td></td><td></td></tr> <tr> <td>12</td><td>12</td><td></td></tr> <tr> <td>89</td><td></td><td>Y</td></tr> <tr> <td>12</td><td></td><td></td></tr> <tr> <td>13</td><td>13</td><td></td></tr> <tr> <td>32</td><td></td><td></td></tr> </table> 1 Mark for these two values, as first instructions 1 Mark for this value, in any row 1 Mark for this value, in any row 1 Mark for this value, after 65, nothing in between 1 Mark for the rest	ACC	Offset	OUTPUT		10		50		2	10			11	11		65					A	11			12	12		89		Y	12			13	13		32			5
ACC	Offset	OUTPUT																																							
	10																																								
50		2																																							
10																																									
11	11																																								
65																																									
		A																																							
11																																									
12	12																																								
89		Y																																							
12																																									
13	13																																								
32																																									

QUESTION

- (e) The program given in **part (d)** is to be translated using a two-pass assembler. The program has been copied here for you.

Label	Instruction
StartProg:	LDV #Offset
	CMP Value
	JPE EndProg
	OUTCH
	LDD Offset
	INC
	STO Offset
	JMP StartProg
EndProg:	END
Offset:	10
	50
	65
	89
	32
Value:	32

On the first pass, the assembly process adds entries to a symbol table.

The following symbol table shows the first five entries, part way through the first pass.

The circular labels show the order in which the assembler made the entries to the symbol table.

Complete the symbol table. Use circular labels to show the order in which the assembler makes the entries.

Symbol table

Symbolic address	Relative address
StartProg (1)	0 (2)
Offset (3)	UNKNOWN (4)
Value (5)	

[6]

MARK SCHEME

4(e)

Mark as follows:

6

Table entries:

1 Mark per bullet, max 4

EndProg

2 · Unknown

9

14

8

Numbering:

1 Mark per bullet, max 2

Relative address of Value is numbered 6

Number given for EndProg is next number in sequence to relative address of Value

All numbers correct – award 2 marks

Symbolic address	Relative address
<u>StartProg</u>	0
Offset	UNKNOWN 9 (1)
Value	UNKNOWN (6) 14 (1)
<u>EndProg</u> (7)	UNKNOWN (8) 8 (9)

QUESTION

- (b) (i) Complete the trace table below for the following assembly language program. This program contains denary values.

100	LDD 800
101	ADD 801
102	STO 802
103	LDD 803
104	CMP 802
105	JPE 107
106	JPN 110
107	STO 802
108	OUT
109	JMP 112
110	LDD 801
111	OUT
112	END
:	
:	
:	
800	40
801	50
802	0
803	90

Selected values from the ASCII character set:

ASCII code	40	50	80	90	100
Character	(	2	P	Z	d

Trace table:

ACC	Memory address				OUTPUT
	800	801	802	803	
	40	50	0	90	

[4]

MARK SCHEME

(b) (i) ONE mark for each correct row.

ACC	Memory address				OUTPUT
	800	801	802	803	
	40	50	0	90	
40					
90			90		
90			90		
					Z

[4]

9608_w16_qp_13_sc_8.b.ii

QUESTION

(ii) There is a redundant instruction in the code in **part (b)(i)**.

State the address of this instruction.

.....[1]

MARK SCHEME

(ii) 107

[1]

QUESTION

(b) Complete the trace table below for the following assembly language program.

800	LDD 810
801	INC ACC
802	STO 812
803	LDD 811
804	ADD 812
805	STO 813
806	END
...	
810	28
811	41
812	0
813	0

Trace table:

ACC	Memory address			
	810	811	812	813
	28	41	0	0

[6]

MARK SCHEME

(b) ONE mark for each correct row.

ACC	Memory address			
	810	811	812	813
	28	41	0	0
28				
29				
			29	
41				
70				
				70

[6]

QUESTION

- (b) (i) Complete the trace table below for the following assembly language program. This program contains denary values.

100	LDD 800
101	ADD 801
102	STO 802
103	LDD 803
104	CMP 802
105	JPE 107
106	JPN 110
107	STO 802
108	OUT
109	JMP 112
110	LDD 801
111	OUT
112	END
:	
:	
:	
800	40
801	50
802	0
803	90

Selected values from the ASCII character set:

ASCII code	40	50	80	90	100
Character	(	2	P	Z	d

Trace table:

ACC	Memory address				OUTPUT
	800	801	802	803	
	40	50	0	90	

[4]

MARK SCHEME

(b) (i) **ONE** mark for each correct row.

ACC	Memory address				OUTPUT
	800	801	802	803	
	40	50	0	90	
40					
90			90		
90			90		
					Z

[4]

9608_w16_qp_11_sc_8.b.ii

QUESTION

(ii) There is a redundant instruction in the code in **part (b)(i)**.

State the address of this instruction.

.....[1]

MARK SCHEME

(ii) **107**

[1]

QUESTION

- 4 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The diagram shows the contents of the index register:

Index register:	1	1	0	0	1	1	0	1
-----------------	---	---	---	---	---	---	---	---

- (a) Show the contents of the index register after the execution of the instruction:

INC IX

Index register:								
-----------------	--	--	--	--	--	--	--	--

[1]

MARK SCHEME

4 (a) 11001110

[1]

QUESTION

(b) Complete the trace table on the opposite page for the following assembly language program.

20	LDX 90
21	DEC ACC
22	STO 90
23	INC IX
24	LDX 90
25	DEC ACC
26	CMP 90
27	JPE 29
28	JPN 31
29	ADD 90
30	OUT
31	ADD 93
32	STO 93
33	OUT
34	END
:	
:	
90	2
91	90
92	55
93	34
IX	2

Selected values from the ASCII character set:

ASCII Code	65	66	67	68	69	70	71	72
Character	A	B	C	D	E	F	G	H

Trace table:

Instruction	Working space	ACC	Memory address				IX	OUTPUT
			90	91	92	93		
			2	90	55	34	2	
20								
21								
22								
23								
24								
25								
26								

[7]

MARK SCHEME

(b)

[7]

Instruction	Working space	ACC	Memory address				IX	OUTPUT
			90	91	92	93		
			2	90	55	34	2	
20		55						
21		54						
22		54						
23							3	
24		34						
25		33						
26								
27								
28								
31		67						
32						67		
33								'C'
34								

One mark each for:

- Instruction 20
- Instructions 21 and 22
- Instruction 23
- Instructions 24 and 25
- Not executing instructions 29 and 30
- Instructions 31 and 32
- Correct output

QUESTION

- 9 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) The diagram shows the current contents of a section of main memory and the index register:

60	0011 0010
61	0101 1101
62	0000 0100
63	1111 1001
64	0101 0101
65	1101 1111
66	0000 1101
67	0100 1101
68	0100 0101
69	0100 0011
...	
1000	0110 1001

Index register:

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

(i) Show the contents of the Accumulator after the execution of the instruction:

LDX 60

Accumulator:

--	--	--	--	--	--	--	--

Show how you obtained your answer.

.....

.....

.....

.....[2]

MARK SCHEME

9 (a) (i) **One mark** for the contents of the accumulator and **one mark** for the reason. [2]

Accumulator contents: 0100 0101

Reason:

Address is 60

Contents of the index register is 8

And $60 + 8 = 68$ in denary gives the address

The contents of which is 0100 0101 in binary.

9608_s16_qp_12_sc_9.a.ii

QUESTION

(ii) Show the contents of the index register after the execution of the instruction:

DEC IX

Index register:

--	--	--	--	--	--	--	--

[1]

MARK SCHEME

(ii) 0000 0111

[1]

QUESTION

(b) Complete the trace table on the opposite page for the following assembly language program.

50	LDD 100
51	ADD 102
52	STO 103
53	LDX 100
54	ADD 100
55	CMP 101
56	JPE 58
57	JPN 59
58	OUT
59	INC IX
60	LDX 98
61	ADD 101
62	OUT
63	END
...	
100	20
101	100
102	1
103	0

IX (Index Register)

Selected values from the ASCII character set:

ASCII Code	118	119	120	121	122	123	124	125
Character	v	w	x	y	z	{		}

Trace table:

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50								
51								
52								
53								
54								
55								

[7]

MARK SCHEME

(b)

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50		20						
51		21						
52						21		
53		100						
54		120						
55								
56								
57								
59							2	
60		20						
61		120						
62								'x'
63								

One mark for each shaded block.

[7]

- Contents of the Accumulator in first 2 lines (instruction addresses 50 and 51)
- Updating address 103 (instruction 52)
- Loading the Accumulator and addition (instructions 53 and 54)
- Not executing instruction 58
- Incrementing the index register (instruction 59)
- Loading the Accumulator and addition (instructions 60 and 61)
- Correct output of 'x' (instruction 62)

QUESTION

- 9 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) The diagram shows the current contents of a section of main memory and the index register:

60	0011 0010
61	0101 1101
62	0000 0100
63	1111 1001
64	0101 0101
65	1101 1111
66	0000 1101
67	0100 1101
68	0100 0101
69	0100 0011
...	
1000	0110 1001

Index register:

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

(i) Show the contents of the Accumulator after the execution of the instruction:

LDX 60

Accumulator:

--	--	--	--	--	--	--	--

Show how you obtained your answer.

.....

.....

.....

.....[2]

MARK SCHEME

9 (a) (i) **One mark** for the contents of the accumulator and **one mark** for the reason. [2]

Accumulator contents: 0100 0101

Reason:

Address is 60

Contents of the index register is 8

And $60 + 8 = 68$ in denary gives the address

The contents of which is 0100 0101 in binary.

9608_s16_qp_11_sc_9.a.ii

QUESTION

(ii) Show the contents of the index register after the execution of the instruction:

DEC IX

Index register:

--	--	--	--	--	--	--	--

[1]

MARK SCHEME

(ii) 0000 0111

[1]

QUESTION

(b) Complete the trace table on the opposite page for the following assembly language program.

50	LDD 100
51	ADD 102
52	STO 103
53	LDX 100
54	ADD 100
55	CMP 101
56	JPE 58
57	JPN 59
58	OUT
59	INC IX
60	LDX 98
61	ADD 101
62	OUT
63	END
...	
100	20
101	100
102	1
103	0

IX (Index Register)

Selected values from the ASCII character set:

ASCII Code	118	119	120	121	122	123	124	125
Character	v	w	x	y	z	{		}

Trace table:

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50								
51								
52								
53								
54								
55								

[7]

MARK SCHEME

(b)

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50		20						
51		21						
52						21		
53		100						
54		120						
55								
56								
57								
59							2	
60		20						
61		120						
62								'x'
63								

One mark for each shaded block.

[7]

- Contents of the Accumulator in first 2 lines (instruction addresses 50 and 51)
- Updating address 103 (instruction 52)
- Loading the Accumulator and addition (instructions 53 and 54)
- Not executing instruction 58
- Incrementing the index register (instruction 59)
- Loading the Accumulator and addition (instructions 60 and 61)
- Correct output of 'x' (instruction 62)

QUESTION

- 7 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load contents of given address to ACC
STO	<address>	Store the contents of ACC at the given address
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
INC	<register>	Add 1 to contents of the register (ACC)
JMP	<address>	Jump to the given address
END		Return control to operating system

The diagram shows the contents of the memory.

Main memory

120	0 0 0 0 1 0 0 1
121	0 1 1 1 0 1 0 1
122	1 0 1 1 0 1 1 0
123	1 1 1 0 0 1 0 0
124	0 1 1 1 1 1 1 1
125	0 0 0 0 0 0 0 1
126	0 1 0 0 0 0 0 1
127	0 1 1 0 1 0 0 1
200	1 0 0 0 1 0 0 0

(a) (i) Show the contents of the Accumulator after execution of the instruction:

LDD 121

Accumulator:

--	--	--	--	--	--	--	--

[1]

MARK SCHEME

7 (a) (i)

Accumulator:

0	1	1	1	0	1	0	1
----------	----------	----------	----------	----------	----------	----------	----------

[1]

QUESTION

(b) Trace the assembly language program using the trace table.

```

300   LDD   321
301   INC
302   STO   323
303   LDI   307
304   INC
305   STO   322
306   END
307   320
  ↗   ↗
320   49
321   36
322   0
323   0

```

Trace table:

Accumulator	Memory address			
	320	321	322	323
	49	36	0	0

[6]

MARK SCHEME

(b) 1 mark for each correct value in the table.

Accumulator	Memory address			
	320	321	322	323
	49	36	0	0
36				
37				
				37
49				
50				
			50	

[6]

Show understanding that a set of instructions are grouped

9608_w19_qp_11_sc_1.c

QUESTION

(c) The following statements describe features of a low-level language.

Complete the statements by writing the appropriate terms in the spaces.

A is a sequence of instructions that are given an identifier. These instructions may need to be executed several times.

A is an instruction that tells the assembler to do something. It is not a program instruction.

The processor's instruction set can be put into several groups. One of these groups is

.....

[3]

MARK SCHEME

1(c)	1 mark for each correctly inserted term A macro is a sequence of instructions that are given an identifier. These instructions may need to be executed several times. A directive is an instruction that tells the assembler to do something. It is not a program instruction. The processor's instruction set can be put into several groups. One of these groups is data movement // input and output // arithmetic operations // jump instructions // compare instructions // modes of addressing.	3
------	---	----------

QUESTION

(b) Assembly language instructions can be grouped by their purpose.

The following table shows four assembly language instructions.

Tick (✓) **one** box in each row to indicate the group each instruction belongs to.

Instruction	Description	Jump instruction	Arithmetic operation	Data movement
LDR #3	Load the number 3 to the Index Register			
ADD #2	Add 2 to the Accumulator			
JPN 22	Move to the instruction at address 22			
DEC ACC	Subtract 1 from the Accumulator			

[3]

MARK SCHEME

4(b)

1 mark for JPN correct
1 mark for both of ADD and DEC correct
1 mark for LDR correct

	Description	Jump instruction	Arithmetic operation	Data movement
LDR #3	Load the number 3 to the Index Register			✓
ADD #2	Add 2 to the Accumulator		✓	
JPN 22	Move to the instruction at address 22	✓		
DEC ACC	Subtract 1 from the Accumulator		✓	

3

QUESTION

(b) A processor’s instruction set can be grouped according to their function. For example, one group is the input and output of data.

Identify **two** other groups of instructions.

1

.....

2

.....

[2]

MARK SCHEME

3(b)	1 mark for each group to max. 2 • Data movement • Arithmetic operations • (Unconditional and conditional) jump instructions • Compare instructions • Modes of addressing	2
------	---	----------

QUESTION

(b) The processor’s instruction set can be grouped according to their function. For example, one group is modes of addressing.

Identify **two** other groups of instructions.

1

.....

2

.....

[2]

MARK SCHEME

5(b)	1 mark per bullet point to max 2 Data movement Input and output of data Arithmetic operations Jump instructions Compare instructions	2
------	---	----------

9608_w18_qp_13_sc_2.b.iv

QUESTION

(iv) Show the result on the general purpose register (X) after the following instruction is run.

INC X

--	--	--	--	--	--	--	--

[1]

MARK SCHEME

2(b)(iv)	<table><tr><td>1</td><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td><td>1</td><td>1</td></tr></table>	1	1	1	1	0	0	1	1	1
1	1	1	1	0	0	1	1			

9608_s16_qp_12_sc_9.a.ii

QUESTION

(ii) Show the contents of the index register after the execution of the instruction:

DEC IX

Index register:

--	--	--	--	--	--	--	--

[1]

MARK SCHEME

(ii) 0000 0111	[1]
----------------	------------

Show understanding of and be able to use different modes of addressing

9608_s21_qp_13_sc_3.b

QUESTION

(b) Identify **two** modes of addressing that are **not** used in the assembly language program given.

1

2 [2]

MARK SCHEME

3(b)	1 mark per bullet point max 2 marks • Direct • Indirect • Relative	2
------	---	----------

9608_s21_qp_12_sc_3.b

QUESTION

(b) The instruction in memory address 50 needs to be changed to use direct addressing to load the contents of the memory location at address 100.

Give the new instruction to replace `LDI 103`.

..... [1]

MARK SCHEME

3(b)	<code>LDD 100</code>	1
------	----------------------	----------

QUESTION

(d) The assembly language program uses direct and indirect addressing.

Identify **two other** modes of addressing used in an assembly language program.

1

2 [2]

MARK SCHEME

3(d)	1 mark each (max 2) • Immediate • Indexed • Relative	2
------	---	----------

QUESTION

(b) The instruction in memory address 100 needs to be changed. It needs to use indirect addressing to load the contents of memory address 200.

Give the new instruction to replace LDD 200.

..... [1]

MARK SCHEME

3(b)	LDD 200	1
------	---------	----------

QUESTION

- 6 A processor has one general purpose register, the Accumulator (ACC), and an Index Register (IX).
- (a) The table gives **three** assembly language instructions for loading data into the ACC. It also identifies the addressing mode used for each instruction.

	Instruction	Addressing mode
A	LDM #193	Immediate
B	LDD 193	Direct
C	LDX 193	Indexed

- (i) State the contents of the Accumulator after each of the instructions **A**, **B** and **C** are run.

A

.....

B

.....

C

.....

[3]

MARK SCHEME

6(a)(i)	1 mark for each correct answer	3
	A: The number 193	
	B: The data in memory location 193	
	C: The data in the memory location found by adding the contents of the IX to 193	

QUESTION

- (ii) Name **two** other addressing modes.

1

2

[2]

MARK SCHEME

6(a)(ii)	1 mark each correct answer • Indirect • Relative	2
----------	--	----------

9608_w19_qp_13_sc_7

QUESTION

7 The following table has descriptions of modes of addressing.

Complete the table by writing the name of the addressing mode for each description.

Addressing mode	Description
	Form the address by adding the given number to a base address. Load the contents of the calculated address to the Accumulator (ACC).
	Load the contents of the address held at the given address to ACC.
	Load the contents of the given address to ACC.
	Form the address from the given address + the contents of the Index Register. Load the contents of the calculated address to ACC.
	Load the given value directly to ACC.

[5]

MARK SCHEME

7	1 mark for each correct addressing mode	5												
<table><tr><th>Addressing mode</th><th>Description</th></tr><tr><td>Relative</td><td>Form the address by adding the given number to a base address. Load the contents of the calculated address to the Accumulator (ACC).</td></tr><tr><td>Indirect</td><td>Load the contents of the address held at the given address to ACC.</td></tr><tr><td>Direct</td><td>Load the contents of the given address to ACC.</td></tr><tr><td>Indexed</td><td>Form the address from the given address + the contents of the Index Register. Load the contents of the calculated address to ACC.</td></tr><tr><td>Immediate</td><td>Load the given value directly to ACC.</td></tr></table>			Addressing mode	Description	Relative	Form the address by adding the given number to a base address. Load the contents of the calculated address to the Accumulator (ACC).	Indirect	Load the contents of the address held at the given address to ACC.	Direct	Load the contents of the given address to ACC.	Indexed	Form the address from the given address + the contents of the Index Register. Load the contents of the calculated address to ACC.	Immediate	Load the given value directly to ACC.
Addressing mode	Description													
Relative	Form the address by adding the given number to a base address. Load the contents of the calculated address to the Accumulator (ACC).													
Indirect	Load the contents of the address held at the given address to ACC.													
Direct	Load the contents of the given address to ACC.													
Indexed	Form the address from the given address + the contents of the Index Register. Load the contents of the calculated address to ACC.													
Immediate	Load the given value directly to ACC.													

QUESTION

4 A program is written in assembly language.

- (a) The op codes `LDM` and `LDD` are used to load a register. The op code `LDM` uses immediate addressing, and the op code `LDD` uses direct addressing.

Describe what happens when the following instructions are run.

`LDM #300`

.....

`LDD 300`

.....

[2]

MARK SCHEME

4(a)	1 mark per bullet	2
	LDM #300 The (denary) number 300 is loaded (into the register) LDD 300 The <u>contents</u> of address 300 are loaded (into the register)	

QUESTION

(c) The following table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDM	#n	Immediate addressing. Load the denary number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the denary number n to IX
STO	<address>	Store contents of ACC at the given address
ADD	<address>	Add the contents of the given address to ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	#n	Compare contents of ACC with denary number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False
JMP	<address>	Jump to the given address
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system

The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are:

Address	Instruction
---------	-------------

50	LDM #0
51	STO 401
52	LDX 300
53	CMP #0
54	JPE 62
55	ADD 400
56	OUT
57	LDD 401
58	INC ACC
59	STO 401
60	INC IX
61	JMP 52
62	END
...	
300	2
301	5
302	0
303	4
...	
400	64
401	
IX	0

ASCII code table (Selected codes only)

ASCII code	Character
65	A
66	B
67	C
68	D
69	E

Trace the program currently in memory using the following trace table. The first instruction has been completed for you.

[illegible]

[8]

MARK SCHEME

3(c)	1 mark per bullet • Storing 0 in 401 (line 51) • Loading memory location 300, value 2 to ACC (line 52) • Adding 64 to ACC to give 66 (line 55) • Outputting B (line 56) • Load 0 (line 57), increment ACC (line 58) and store 1 in 401 (line 59) • Incrementing IX (line 60) • Loading 5 (line 52), adding 64 (line 55), outputting E (line 56) loading 1 (line 57), incrementing ACC (line 58), storing 2 in 401 (line 59) and incrementing IX (line 60) • Load 0 (line 52) <u>and end</u>	8
------	--	----------

QUESTION

- (c) The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC), and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDM	#n	Immediate addressing. Load the denary number n to ACC.
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the denary number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
CMP	#n	Compare contents of ACC with denary number n.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The current contents of the main memory, Index Register (IX) and selected values from the ASCII character set are:

Address Instruction

20	LDM #0
21	STO 300
22	CMP #0
23	JPE 28
24	LDX 100
25	ADD 301
26	OUT
27	JMP 30
28	LDX 100
29	OUT
30	LDD 300
31	INC ACC
32	STO 300
33	INC IX
34	CMP #2
35	JPN 22
36	END
...	
100	65
101	67
102	69
103	69
104	68
...	
300	
301	33
IX	0

ASCII code table (Selected codes only)

ASCII Code	Character
65	A
66	B
67	C
68	D
69	E
97	a
98	b
99	c
100	d
101	e

Trace the program currently in memory using the following trace table. The first instruction has been completed for you.

[illegible]

[8]

MARK SCHEME

5(c)	1 mark per bullet point - Storing 0 in 300 (line 21) - Loading 65 (line 28) - Outputting A (line 29) - Loading 0 (line 30), incrementing ACC (line 31) and storing in 300 (line 32) - Incrementing IX (line 33) - Loading 67 (line 24) and adding 33 (line (25) - Outputting d (line 26) - Loading 1 (line 30), incrementing ACC (line 31), storing in 300 (line 32) and incrementing IX (line 33)	8
------	--	----------

QUESTION

- 2 The following table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) State what is meant by **relative addressing** and **indexed addressing**.

Relative addressing

.....

.....

Indexed addressing

.....

.....

[2]

MARK SCHEME

2(a)	1 mark per bullet point Relative addressing: - A number (the offset) is added to the base address to give the actual address Indexed addressing: - The contents of the index register are added to operand to give the actual address	2
------	---	----------

9608_w18_qp_12_sc_3.a.i

QUESTION

- 3** The following table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) (i)** State what is meant by **absolute addressing** and **symbolic addressing**.

Absolute addressing

.....

Symbolic addressing

.....

[2]

MARK SCHEME

3(a)(i)	1 mark per bullet point Absolute addressing: - The operand is a numeric address // The numeric address is given // referring directly to a memory location Symbolic addressing: - The operand is a word/symbol // A word/symbol represents the memory location/address	2
---------	--	----------

9608_w18_qp_12_sc_3.a.ii

QUESTION

(ii) Give an example of an `ADD` instruction using both absolute addressing and symbolic addressing.

Absolute addressing

Symbolic addressing [2]

MARK SCHEME

3(a)(ii)	1 mark per example Absolute addressing: For example, <code>ADD 230</code> Symbolic addressing: For example, <code>ADD num1</code>	2
----------	--	----------

9608_w18_qp_12_sc_3.b.i

QUESTION

(b) (i) State what is meant by **indexed addressing** and **immediate addressing**.

Indexed addressing

.....

Immediate addressing

..... [2]

MARK SCHEME

3(b)(i)	1 mark per bullet point Indexed addressing: - The address to be used is formed by: operand + the contents of the Index Register (IX) Immediate addressing: - The operand is not an address // the operand is the actual value to be loaded	2
---------	--	----------

9608_w18_qp_12_sc_3.b.ii

QUESTION

(ii) Give an example of an instruction that uses: Indexed addressing Immediate addressing [2]
--

MARK SCHEME

3(b)(ii)	1 mark per example Indexed: For example, LDX 20 Immediate: For example, ADD #20	2
----------	--	----------

QUESTION

- 4 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the Index Register. Copy the contents of this calculated address to ACC.
LDR	#n	Immediate addressing. Load the number n to IX.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) (i) State what is meant by **direct addressing** and **indirect addressing**.

Direct addressing

.....

Indirect addressing

.....

[2]

MARK SCHEME

4(a)(i)	1 mark for each type of addressing Direct addressing The operand is the address where the data is stored Indirect addressing The operand is an address, that address holds another address where the data is stored	2
---------	---	----------

QUESTION

- (ii) Explain how the instruction `ADD 20` can be interpreted as either direct or indirect addressing.

Direct addressing

.....

Indirect addressing

.....

[2]

MARK SCHEME

4(a)(ii)	1 mark per bullet point Direct addressing: 20 is the address of the data Indirect addressing: 20 is an address which holds the address where the data is stored	2
----------	--	----------

QUESTION

- (b) The assembly language instructions in the following table use either symbolic addressing or absolute addressing.

Tick (✓) **one** box in each row to indicate whether the instruction uses symbolic or absolute addressing.

Instruction	Symbolic	Absolute
<code>ADD 90</code>		
<code>CMP found</code>		
<code>STO 20</code>		

[2]

MARK SCHEME

4(b)	1 mark for 1 correct tick 2 marks for 3 correct ticks	2												
<table><tr><th>Instruction</th><th>Symbolic</th><th>Absolute</th></tr><tr><td>ADD 90</td><td></td><td>✓</td></tr><tr><td>CMP found</td><td>✓</td><td></td></tr><tr><td>STO 20</td><td></td><td>✓</td></tr></table>			Instruction	Symbolic	Absolute	ADD 90		✓	CMP found	✓		STO 20		✓
Instruction	Symbolic	Absolute												
ADD 90		✓												
CMP found	✓													
STO 20		✓												

9608_s17_qp_13_sc_4.b

QUESTION

503

502

(b) Each machine code instruction is encoded as 16-bits (8-bit op code followed by an 8-bit operand).

Write the machine code for the following instructions:

LDM #17

LDX #97

[3]

MARK SCHEME

4(b)																	3
	0 0 0 0 0 0 0 1								0 0 0 1 0 0 0 1								
	0 0 0 0 0 1 1 0								0 1 1 0 0 0 0 1								
	Both correct op codes								1								
	Operand 0001 0001								1								
	Operand 0110 0001								1								

9608_s17_qp_13_sc_4.c

QUESTION

- (c) Using an 8-bit operand, state the maximum number of memory locations, in denary, that can be directly addressed.

.....[1]

MARK SCHEME

4(c)	256	1
------	-----	---

9608_s17_qp_12_sc_5.b

QUESTION

357	22
358	

- (b) Each machine code instruction is encoded as 16 bits (8-bit op code followed by an 8-bit operand).

Write the machine code for these instructions:

LDM #67

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

LDX #7

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

[3]

MARK SCHEME

5(b)	<table><tr><th colspan="8">Op code</th><th colspan="8">Operand</th></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr></table>	Op code								Operand								0	0	0	1	0	0	1	0	0	1	0	0	0	0	1	1	0	0	0	1	0	1	0	1	0	0	0	0	0	1	1	1	3
Op code								Operand																																										
0	0	0	1	0	0	1	0	0	1	0	0	0	0	1	1																																			
0	0	0	1	0	1	0	1	0	0	0	0	0	1	1	1																																			
	Both correct op codes	1																																																
	Operand 0100 0011	1																																																
	Operand 0000 0111	1																																																

QUESTION

503

502

(b) Each machine code instruction is encoded as 16-bits (8-bit op code followed by an 8-bit operand).

Write the machine code for the following instructions:

LDM #17

LDX #97

[3]

MARK SCHEME

4(b)	0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 0 0 1 1 0 0 1 1 0 0 0 0 1 Both correct op codes Operand 0001 0001 Operand 0110 0001 1 1 1	3
------	---	---

QUESTION

(c) Using an 8-bit operand, state the maximum number of memory locations, in denary, that can be directly addressed.

.....[1]

MARK SCHEME

4(c)	256	1
------	-----	---

QUESTION

- 8 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
CMP	<address>	Compare contents of ACC with contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The diagram shows the contents of the main memory:

Main memory	
800	0110 0100
801	0111 1100
802	1001 0111
803	0111 0011
804	1001 0000
805	0011 1111
806	0000 1110
807	1110 1000
808	1000 1110
809	1100 0010
:	
:	
2000	1011 0101

- (a) (i) Show the contents of the Accumulator after execution of the instruction:

LDD 802

Accumulator:

--	--	--	--	--	--	--	--

[1]

MARK SCHEME

8 (a) (i)

Accumulator:

1	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---

[1]

9608_w16_qp_13_sc_8.a.ii

QUESTION

(ii) Show the contents of the Accumulator after execution of the instruction:

LDX 800

Index Register:

0	0	0	0	1	0	0	1
---	---	---	---	---	---	---	---

Accumulator:

--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

.....

[3]

MARK SCHEME

(ii) **One mark** for answer and **two marks** for explanation

Accumulator:

1	1	0	0	0	0	1	0
---	---	---	---	---	---	---	---

- Index Register contains $1001 = 9$
- $800 + 9 = 809$

[3]

QUESTION

- 5 The table shows assembly language instructions for a processor that has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op Code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Index addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC.
STO	<address>	Store the contents of ACC at the given address.
INC	<register>	Add 1 to contents of the register (ACC or IX).
ADD	<address>	Add the contents of the given address to the ACC.
END		Return control to the operating system.

The diagram shows the contents of a section of main memory:

Main memory	
100	0000 0010
101	1001 0011
102	0111 0011
103	0110 1011
104	0111 1110
105	1011 0001
106	0110 1000
107	0100 1011
...	
200	1001 1110

- (a) (i) Show the contents of the Accumulator after the execution of the instruction:

LDD 102

ACC:

--	--	--	--	--	--	--	--

[1]

MARK SCHEME

5 (a) (i)

0	1	1	1	0	0	1	1
---	---	---	---	---	---	---	---

[1]

9608_w16_qp_12_sc_5.a.ii

QUESTION

(ii) Show the contents of the Accumulator after the execution of the instruction:

LDX 101

IX:	0	0	0	0	0	1	0	0
-----	---	---	---	---	---	---	---	---

ACC:								
------	--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

.....

[2]

MARK SCHEME

(ii) **ONE** mark for Accumulator contents, **ONE** mark for the explanation.

1	0	1	1	0	0	0	1
---	---	---	---	---	---	---	---

- Index Register holds the value 4; $101 + 4 = 105$ so load data from address 105

[2]

QUESTION

(iii) Show the contents of the Accumulator after the execution of the instruction:

LDI 103

ACC:

--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

.....
.....
.....
.....
.....
.....[3]

MARK SCHEME

(iii) **ONE** mark for Accumulator contents, **TWO** marks for the explanation.

0	1	0	0	1	0	1	1
---	---	---	---	---	---	---	---

- Memory address 103 contains the value 107
- So address 107 is the address from which to load the data

[3]

QUESTION

- 8 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
CMP	<address>	Compare contents of ACC with contents of <address>
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to the screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

The diagram shows the contents of the main memory:

Main memory	
800	0110 0100
801	0111 1100
802	1001 0111
803	0111 0011
804	1001 0000
805	0011 1111
806	0000 1110
807	1110 1000
808	1000 1110
809	1100 0010
:	
:	
2000	1011 0101

- (a) (i) Show the contents of the Accumulator after execution of the instruction:

LDD 802

Accumulator:

--	--	--	--	--	--	--	--

[1]

MARK SCHEME

8 (a) (i)

Accumulator:

1	0	0	1	0	1	1	1
---	---	---	---	---	---	---	---

[1]

9608_w16_qp_11_sc_8.a.ii

QUESTION

(ii) Show the contents of the Accumulator after execution of the instruction:

LDX 800

Index Register:

0	0	0	0	1	0	0	1
---	---	---	---	---	---	---	---

Accumulator:

--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

.....

[3]

MARK SCHEME

(ii) **One mark** for answer and **two marks** for explanation

Accumulator:

1	1	0	0	0	0	1	0
---	---	---	---	---	---	---	---

- Index Register contains $1001 = 9$
- $800 + 9 = 809$

[3]

QUESTION

(b) Complete the trace table on the opposite page for the following assembly language program.

20	LDX 90
21	DEC ACC
22	STO 90
23	INC IX
24	LDX 90
25	DEC ACC
26	CMP 90
27	JPE 29
28	JPN 31
29	ADD 90
30	OUT
31	ADD 93
32	STO 93
33	OUT
34	END
:	
:	
90	2
91	90
92	55
93	34
IX	2

Selected values from the ASCII character set:

ASCII Code	65	66	67	68	69	70	71	72
Character	A	B	C	D	E	F	G	H

Trace table:

Instruction	Working space	ACC	Memory address				IX	OUTPUT
			90	91	92	93		
			2	90	55	34	2	
20								
21								
22								
23								
24								
25								
26								

[7]

MARK SCHEME

(b)

[7]

Instruction	Working space	ACC	Memory address				IX	OUTPUT
			90	91	92	93		
			2	90	55	34	2	
20		55						
21		54						
22		54						
23							3	
24		34						
25		33						
26								
27								
28								
31		67						
32						67		
33								'C'
34								

One mark each for:

- Instruction 20
- Instructions 21 and 22
- Instruction 23
- Instructions 24 and 25
- Not executing instructions 29 and 30
- Instructions 31 and 32
- Correct output

QUESTION

- 9 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) The diagram shows the current contents of a section of main memory and the index register:

60	0011 0010
61	0101 1101
62	0000 0100
63	1111 1001
64	0101 0101
65	1101 1111
66	0000 1101
67	0100 1101
68	0100 0101
69	0100 0011
...	
1000	0110 1001

Index register:

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

(i) Show the contents of the Accumulator after the execution of the instruction:

LDX 60

Accumulator:

--	--	--	--	--	--	--	--

Show how you obtained your answer.

.....
.....
.....
.....[2]

MARK SCHEME

9 (a) (i) One mark for the contents of the accumulator and **one mark** for the reason. **[2]**

Accumulator contents: 0100 0101

Reason:

Address is 60

Contents of the index register is 8

And $60 + 8 = 68$ in denary gives the address

The contents of which is 0100 0101 in binary.

QUESTION

(b) Complete the trace table on the opposite page for the following assembly language program.

50	LDD 100
51	ADD 102
52	STO 103
53	LDX 100
54	ADD 100
55	CMP 101
56	JPE 58
57	JPN 59
58	OUT
59	INC IX
60	LDX 98
61	ADD 101
62	OUT
63	END
...	
100	20
101	100
102	1
103	0

IX (Index Register)

Selected values from the ASCII character set:

ASCII Code	118	119	120	121	122	123	124	125
Character	v	w	x	y	z	{		}

Trace table:

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50								
51								
52								
53								
54								
55								

[7]

MARK SCHEME

(b)

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50		20						
51		21						
52						21		
53		100						
54		120						
55								
56								
57								
59							2	
60		20						
61		120						
62								'x'
63								

One mark for each shaded block.

[7]

- Contents of the Accumulator in first 2 lines (instruction addresses 50 and 51)
- Updating address 103 (instruction 52)
- Loading the Accumulator and addition (instructions 53 and 54)
- Not executing instruction 58
- Incrementing the index register (instruction 59)
- Loading the Accumulator and addition (instructions 60 and 61)
- Correct output of 'x' (instruction 62)

QUESTION

- 9 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC) and an index register (IX).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load the contents of the given address to ACC.
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC.
STO	<address>	Store contents of ACC at the given address.
ADD	<address>	Add the contents of the given address to ACC.
INC	<register>	Add 1 to the contents of the register (ACC or IX).
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX).
CMP	<address>	Compare contents of ACC with contents of <address>.
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True.
JPN	<address>	Following a compare instruction, jump to <address> if the compare was False.
JMP	<address>	Jump to the given address.
OUT		Output to screen the character whose ASCII value is stored in ACC.
END		Return control to the operating system.

- (a) The diagram shows the current contents of a section of main memory and the index register:

60	0011 0010
61	0101 1101
62	0000 0100
63	1111 1001
64	0101 0101
65	1101 1111
66	0000 1101
67	0100 1101
68	0100 0101
69	0100 0011
...	
1000	0110 1001

Index register:

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

(i) Show the contents of the Accumulator after the execution of the instruction:

LDX 60

Accumulator:

--	--	--	--	--	--	--	--

Show how you obtained your answer.

.....
.....
.....
.....[2]

MARK SCHEME

9 (a) (i) One mark for the contents of the accumulator and **one mark** for the reason. **[2]**

Accumulator contents: 0100 0101

Reason:

Address is 60

Contents of the index register is 8

And $60 + 8 = 68$ in denary gives the address

The contents of which is 0100 0101 in binary.

QUESTION

(b) Complete the trace table on the opposite page for the following assembly language program.

50	LDD 100
51	ADD 102
52	STO 103
53	LDX 100
54	ADD 100
55	CMP 101
56	JPE 58
57	JPN 59
58	OUT
59	INC IX
60	LDX 98
61	ADD 101
62	OUT
63	END
...	
100	20
101	100
102	1
103	0

IX (Index Register)

Selected values from the ASCII character set:

ASCII Code	118	119	120	121	122	123	124	125
Character	v	w	x	y	z	{		}

Trace table:

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50								
51								
52								
53								
54								
55								

[7]

MARK SCHEME

(b)

Instruction address	Working space	ACC	Memory address				IX	OUTPUT
			100	101	102	103		
			20	100	1	0	1	
50		20						
51		21						
52						21		
53		100						
54		120						
55								
56								
57								
59							2	
60		20						
61		120						
62								'x'
63								

One mark for each shaded block.

[7]

- Contents of the Accumulator in first 2 lines (instruction addresses 50 and 51)
- Updating address 103 (instruction 52)
- Loading the Accumulator and addition (instructions 53 and 54)
- Not executing instruction 58
- Incrementing the index register (instruction 59)
- Loading the Accumulator and addition (instructions 60 and 61)
- Correct output of 'x' (instruction 62)

QUESTION

- 7 The table shows assembly language instructions for a processor which has one general purpose register, the Accumulator (ACC).

Instruction		Explanation
Op code	Operand	
LDD	<address>	Direct addressing. Load contents of given address to ACC
STO	<address>	Store the contents of ACC at the given address
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
INC	<register>	Add 1 to contents of the register (ACC)
JMP	<address>	Jump to the given address
END		Return control to operating system

The diagram shows the contents of the memory.

Main memory

120	0 0 0 0 1 0 0 1
121	0 1 1 1 0 1 0 1
122	1 0 1 1 0 1 1 0
123	1 1 1 0 0 1 0 0
124	0 1 1 1 1 1 1 1
125	0 0 0 0 0 0 0 1
126	0 1 0 0 0 0 0 1
127	0 1 1 0 1 0 0 1
200	1 0 0 0 1 0 0 0

(a) (i) Show the contents of the Accumulator after execution of the instruction:

LDD 121

Accumulator:

--	--	--	--	--	--	--	--

[1]

MARK SCHEME

7 (a) (i)

Accumulator:

0	1	1	1	0	1	0	1
---	---	---	---	---	---	---	---

[1]

9608_s15_qp_13_sc_7.a.ii

QUESTION

(ii) Show the contents of the Accumulator after execution of the instruction:

LDI 124

Accumulator:

--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

.....

.....

.....

..... [3]

MARK SCHEME

(ii)

Accumulator:

0	1	1	0	1	0	0	1
---	---	---	---	---	---	---	---

[1]

explanation

- content of 124 is 0 1 1 1 1 1 1 1
- this is equivalent to 127
- contents of 127 are 0 1 1 0 1 0 0 1

[2]

QUESTION

(iii) Show the contents of the Accumulator after execution of the instruction:

LDX 120

Index Register:	0	0	0	0	0	1	1	0
-----------------	---	---	---	---	---	---	---	---

Accumulator:								
--------------	--	--	--	--	--	--	--	--

Explain how you arrived at your answer.

.....

 [3]

MARK SCHEME

(iii)

Accumulator:	0	1	0	0	0	0	0	1
--------------	---	---	---	---	---	---	---	---

[1]

explanation

- index register value = 6
- $120 + 6 = 126$
- contents of 126 placed in the accumulator

[2]

QUESTION

- 3 Five modes of addressing and five descriptions are shown below.

Draw a line to connect **each** mode of addressing to its correct description.

Mode of addressing

direct

immediate

indexed

indirect

relative

Description

the operand is the address of the address of the value to be used

the operand is the address of the value to be used

the operand is the offset from the current address where the value to be used is stored

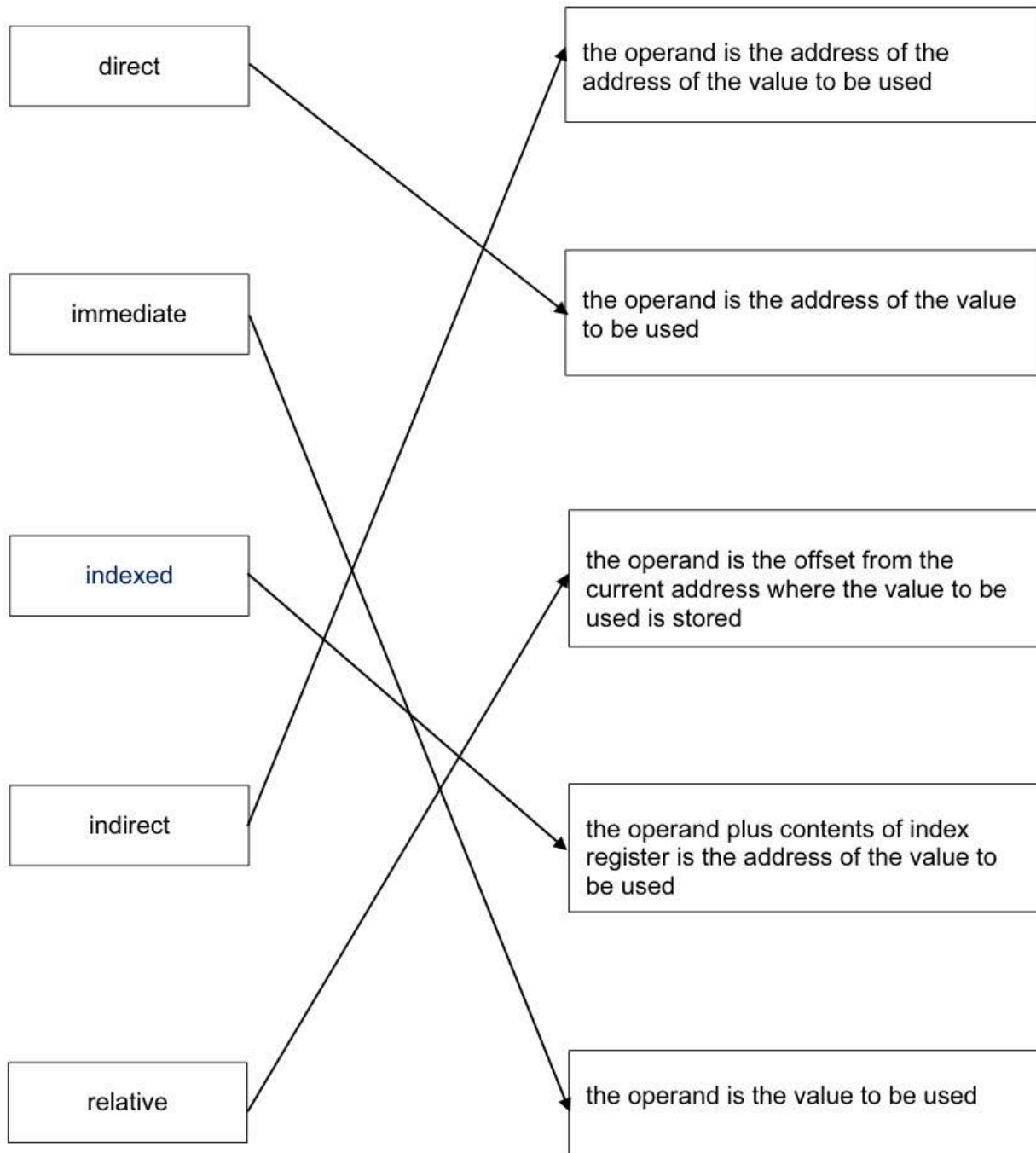
the operand plus the contents of the index register is the address of the value to be used

the operand is the value to be used

[4]

MARK SCHEME

3



[4]

QUESTION

- 3 Five modes of addressing and five descriptions are shown below.

Draw a line to connect **each** mode of addressing to its correct description.

Mode of addressing

direct

immediate

indexed

indirect

relative

Description

the operand is the address of the address of the value to be used

the operand is the address of the value to be used

the operand is the offset from the current address where the value to be used is stored

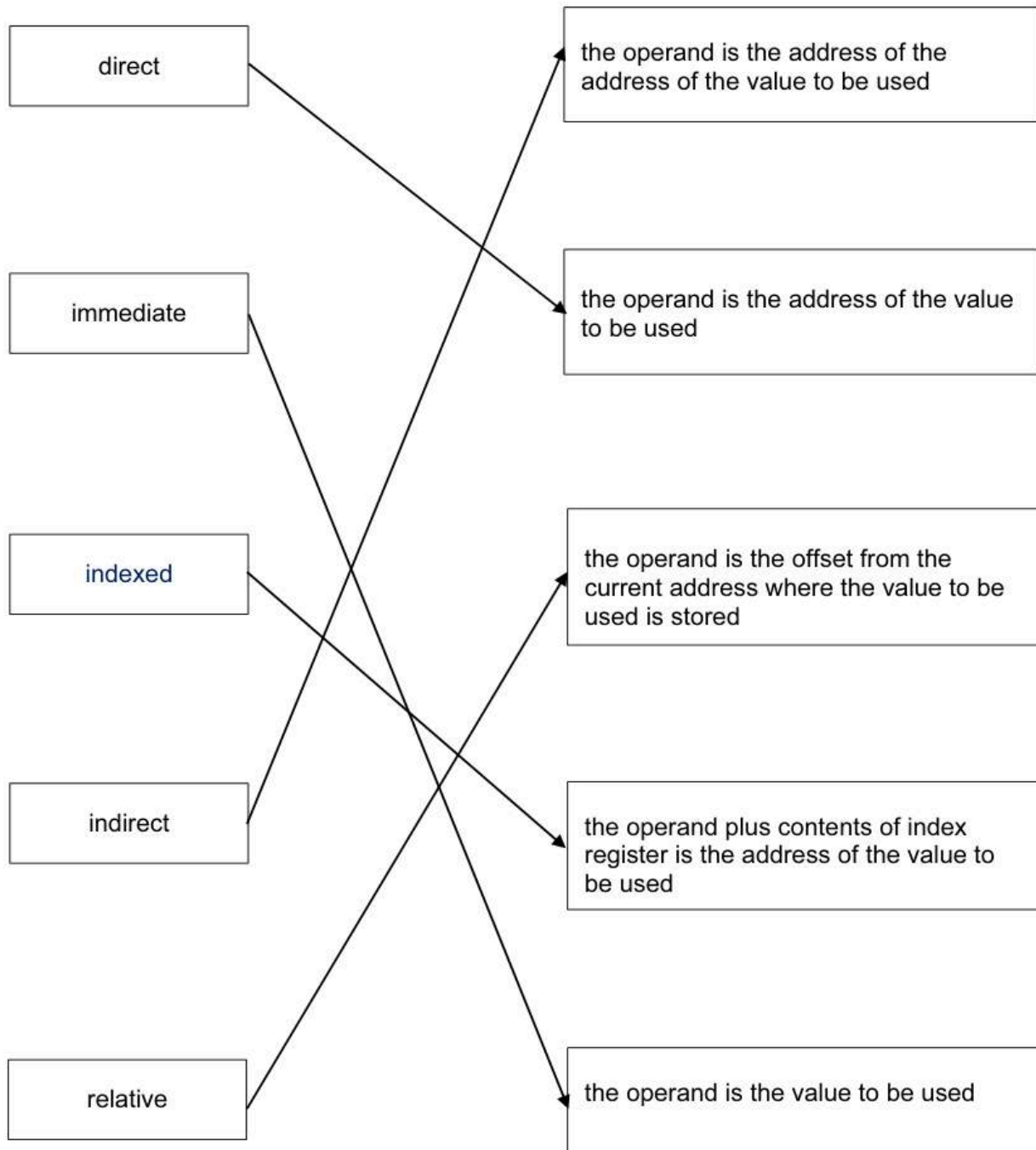
the operand plus the contents of the index register is the address of the value to be used

the operand is the value to be used

[4]

MARK SCHEME

3



[4]